

Automatiser les tests avec Postman.

Formation pratique 2 jours

Didier Curvier MALEMBE

1. Introduction à Postman	2
a. Installation et prise en main	2
b. Présentation de l'interface	2
c. Les différents protocoles pris en charge	2
d. Lancer une première requêtes	2
2. Fondamentaux de API & du Protocole HTTP	3
a. Le protocole HTTP : Comprendre les messages HTTP	3
b. Architecture REST : Principes, ressources, endpoints et structure des URLs	3
c. Format JSON : Structure des données échangées entre client et serveur.	3
d. Documentation openAPI/swagger : Lire et Interpréter une specification pour en déduire les requêtes à construire	3
e. Travaux-pratiques :	3
3. Structurer les requêtes avec Postman	4
a. Requetes CRUD : Création des requêtes GET, POST, PUT, DELETE, PATCH sur une API REST.	4
b. Paramètres de requêtes : Utilisation des query parameters, path parameters et headers personnalisés.	4
c. Corps de requête : envoie de données en JSON, form-data ou autres formats.	4
d. Authentification : configuration de API KEY, Bearer-Token, Basic Auth selon l'API.	4
e. Import de requêtes : Récupération de requêtes depuis un navigateur ou un fichier de collection.	4
f. Travaux pratiques :	4
4. Utilisation des variables dans Postman	5
a. Variables d'environnement : Isoler les configurations spécifiques à chaque contexte (dev, recette, production)	5
b. Variable de collection et variable globale : portée et priorité de résolution des différents scopes.	5
c. Données dynamiques : Générer des valeurs aléatoires et horodatées avec les variables intégrées de Postman.	5
d. Variabilisation des requêtes : Remplacer les valeurs en dur par des références de variables pour rendre les collections portables.	5
e. Travaux pratiques :	5
5. Scripts et automatisisation	5
a. Écriture de scripts Pre-request pour préparer et valider le contexte d'une requête.	5
b. Ecriture Scripts post response de test pour valider les réponses (status code, body, headers, temps de réponse)	5
c. utiliser la console Postman pour inspecter les requêtes, réponses.	5
d. Chaînage de requêtes.	5
e. Travaux pratiques :	5
6. Newman pour automatiser l'exécution des tests	6
a. Postman collection runner	6
b. Utilisation de moniteurs	6
c. Automatiser ses test avec Newman et Postman	6
d. Automatiser l'exécution de test dans une CI avec Gitlab CI	6

e. Générer des rapports

6

f. Travaux pratiques :

6

1. Introduction à Postman

Qu'est ce que Postman ?

Postman est un outil de développement et de test **d'API** qui simplifie le processus d'envoi de requêtes, l'analyse des réponses et la gestion des **API**. Il offre une interface conviviale pour créer et exécuter des appels **d'API**, facilitant ainsi les interactions avec les services web.

Postman transforme le développement d'API – autrefois fragmenté et dépendant de multiples outils – en un processus unifié et collaboratif couvrant la **conception**, les **tests**, la **documentation** et la **surveillance**.

Postman permet aux organisations de créer des **API** prêtes pour l'intelligence artificielle, utilisées à la fois par des équipes humaines et des agents automatisés en maintenant les contrôles de sécurité, les cadre de conformité et les standards de gouvernance requis par les entreprises.

a. Présentation de l'interface

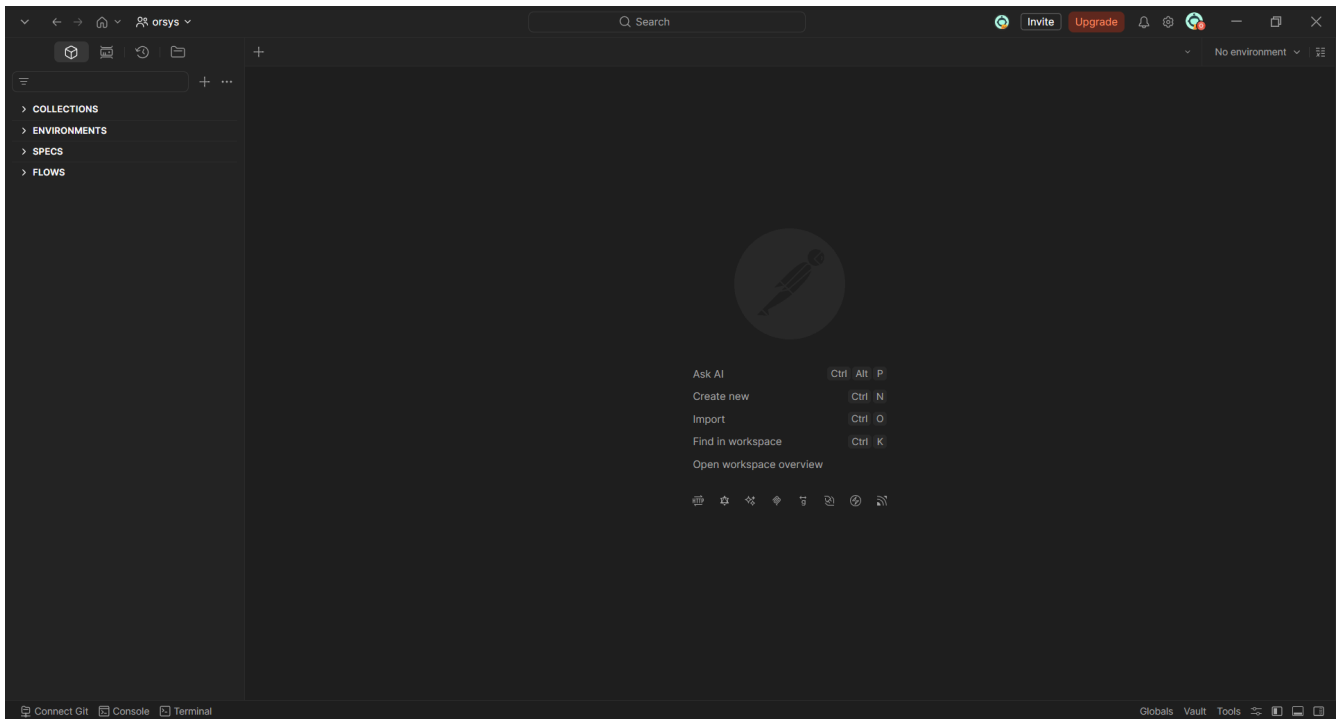
L'interface de Postman organise les projets en workspace.

On appelle workspace un ensemble d'outils Postman pour travailler et collaborer en équipe. Ces outils permettent la gestion des API de la conception au monitoring.

Les workspaces dans postman sont de 3 types :

- **Workspaces Personnel : Idéal pour l'apprentissage ou les tests.**
- **Workspaces Team : Partage des ressources en équipe – Collaboration en temps réel et idéal pour l'usage en entreprise.**
- **Workspace public : Les ressources du projet sont visibles par tout le monde – permet l'exposition publique d'une API.**

Les workspaces favorisent une meilleure structuration des projets/API grâce à la centralisation des ressources d'une API.



b. Les différents protocoles pris en charge

Les principaux protocoles supportés sur Postman sont les suivants :

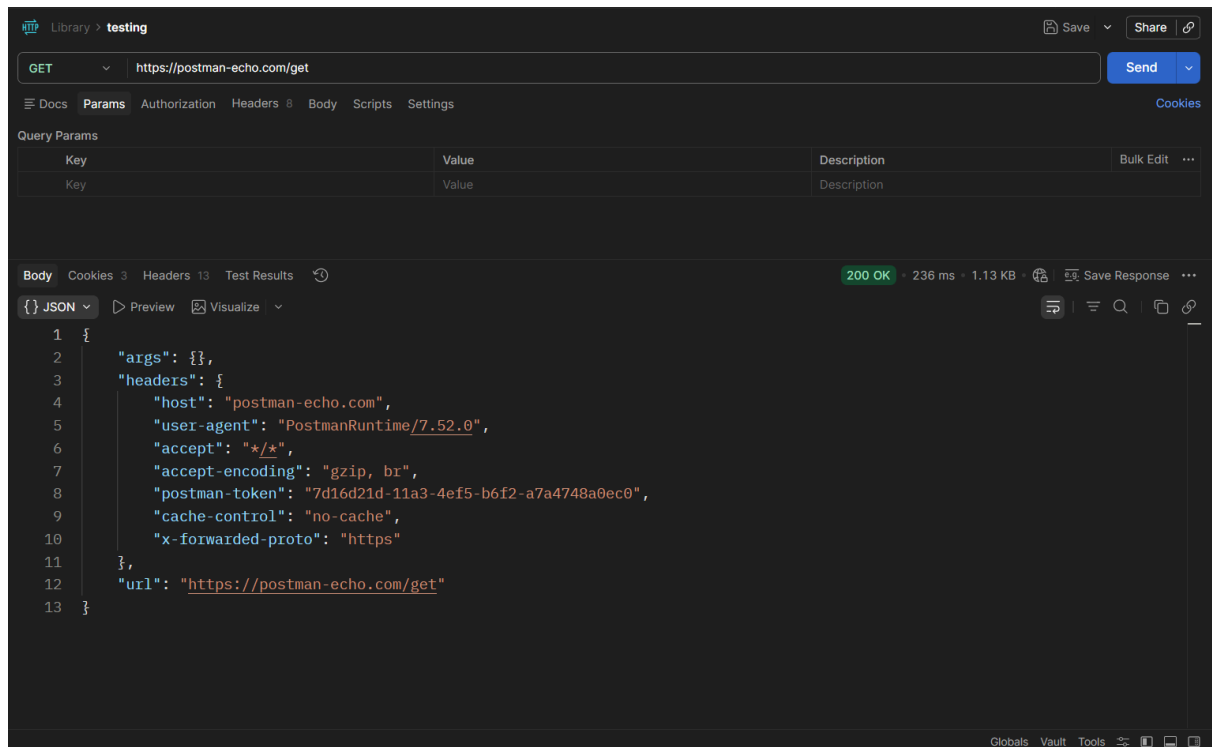
- **HTTP/HTTPS**
- **WebSocket**
- **gRPC**
- **GraphQL**
- **SOAP**

Les plus couramment utilisés sont le **HTTP/HTTPS** et le **SOAP**.

c. Lancer une première requêtes

Ouvrez postman et testez la requête suivante :

<https://postman-echo.com/get>



2. Fondamentaux de API & du Protocole HTTP

a. Le protocole HTTP : Comprendre les messages HTTP

Rappels TCP, HTTP et persistance

- **TCP (Transmission Control Protocol)** est un protocole de transport garantissant la transmission fiable des données entre deux machines.
- **HTTP (HyperText Transfer Protocol)** est un protocole applicatif basé sur TCP, utilisé pour la communication entre clients (navigateur, API) et serveurs web.

HTTP utilise **TCP** comme protocole de transport sous-jacent, une connexion **TCP** est établie avant l'échange des requêtes **HTTP**.

HTTP persistant vs NonPersistant

- **HTTP 1.0** : Par défaut, chaque requête ouvre et ferme une connexion TCP, ce qui est coûteux en termes de performance.
- **HTTP 1.1 (Persistant)** : Introduit la persistance avec Connection: keep-alive, permettant de réutiliser la connexion TCP pour plusieurs requêtes.

Les PDU (Protocol Data Unit) HTTP : GET - POST - PUT - DELETE - HEAD - OPTION

Les méthodes HTTP

Il existe 11 méthodes **HTTP**

GET: Récupère une ressource sans modifier l'état du serveur.

POST: Envoie des données au serveur pour créer une nouvelle ressource.

PUT: Met à jour ou crée une ressource spécifique avec un contenu fourni.

DELETE: Supprime une ressource identifiée par une URL.

HEAD: Similaire à GET, mais ne retourne que les en-têtes HTTP, sans le corps.

TRACE: Diagnostique le chemin d'une requête HTTP en renvoyant l'intégralité de la requête reçue par le serveur.

OPTION : Permet de savoir à quelles méthodes va répondre le serveur.

Deux approches sont possibles :

OPTIONS * HTTP/1.1 : On demande toutes les méthodes autorisées pour l'ensemble des ressources du serveur.

OPTIONS /maRessource HTTP/1.1 : On demande les méthodes autorisées pour une ressource spécifique

Les En-tête HTTP

Voici une liste non-exhaustive d'en-tête **HTTP** intéressantes dans le cadre d'implémentation **REST**

Accept-Charset, Accept-Encoding

Ces en-tête permettent de savoir que le serveur et le client peuvent communiquer.

Accept-Charset : Indique le ou les jeux de caractères, supportés par le client et séparés par des (,).

* signifie que n'importe quel caractère est accepté.

Accept-Charset : ISO-8859-1,UTF-8

Accept-Charset : *

Accept-Encoding : indique les encodages supportés par l'application séparés par des (,). Les valeurs usuelles sont compress et gzip. * signifie que n'importe quel type d'encodage est accepté.

Accept-Encoding : compress, gzip

Accept-Encoding : *

Accept-Language et Content-Language

L'en-tête **Accept-Language** est envoyé dans une requête, tandis que **Content-Language** est envoyé dans la réponse. Ces deux en-tête désignent la langue utilisée dans le contenu.

Accept-Language = fr-fr

Content-Language = fr-fr

Age

L'en-tête Age est utilisé dans une réponse, il indique l'âge d'une ressource en seconde. Obligatoire lorsqu'un serveur sert du contenu depuis le cache.

Age = 50

Allow

L'en-tête est ajouté dans une réponse et indique la (les) méthode (s) autorisée par le serveur.

Si l'application appelante appelle une autre méthode, le serveur doit lui fournir le code 405 (*Method Not Allowed*).

Allow : GET, POST, DELETE

Accept-Type & Content-Type

Ces en-têtes sont parmi les plus importants lorsqu'une application fournit des services REST dont les réponses peuvent être dans plusieurs formats.

Accept-Type :

L'en-tête Accept-Type se trouve dans la requête, il définit le ou les types de données que l'application peut gérer.

Accept : * / *

Accept : application/json

Accept : text/*, text/plain, text/html

Accept : application/xml

Si le serveur n'est pas capable de retourner l'un des formats demandés, il devra fournir une réponse de code 406 (Not Acceptable).

Content-Type :

L'en-tête Content-Type positionné dans la réponse indique le type de données retourné par l'application

Content-Type : application/json

Content-Type : text/html; charset = UTF-8

Authorization

L'en-tête est ajouté à une requête lorsque l'accès à une ressource nécessite une authentification.

Authorization : Basic : ezrorijfjhdyyeol==

Code de statut HTTP (1xx - 5xx)

1xx (Informational) : La requête est en cours de traitement.

2xx (Succès) : La requête a réussi (200 OK, 201 Created).

Quoi que le client ait tenté de faire, cela a réussi jusqu'au point où la réponse a été envoyée. Gardez à l'esprit qu'un statut comme 202 Accepted ne dit rien sur le résultat réel ; il indique simplement que la requête a été acceptée et est en cours de traitement de manière asynchrone.

3xx (Redirection) : La ressource a été déplacée (301 Moved Permanently, 302 Found). Il s'agit ici de rediriger l'application appelante vers un autre emplacement pour accéder à la ressource réelle. Les plus connus de ces statuts sont 303 See Other et 301 Moved Permanently, couramment utilisés sur le web pour rediriger un navigateur vers une autre URL.

4xx (Erreur client) : Erreur côté client (400 Bad Request, 401 Unauthorized, 404 Not Found). Avec ces codes de statut, nous indiquons que le client a fait quelque chose d'invalidé et qu'il doit corriger la requête avant de la renvoyer.

5xx (Erreur serveur) : Erreur côté serveur (500 Internal Server Error, 503 Service Unavailable).

Avec ces codes de statut, nous indiquons qu'une erreur s'est produite au niveau du service. Par exemple, une connexion à la base de données a échoué. En général, une application cliente peut réessayer la requête. Le serveur peut même spécifier quand le client est autorisé à réessayer la commande en utilisant l'en-tête HTTP Retry-After.

Voici une liste non exhaustive de codes statut et leur contexte d'utilisation.

200 – Tout est OK de manière générique

201 – Création réussie d'une ressource

202 – Requête acceptée mais en cours de traitement asynchrone (pour une vidéo : encodage, pour une image : redimensionnement, etc.)

400 – Mauvaise requête (devrait en principe être réservé à une syntaxe invalide, mais certains l'utilisent aussi pour les erreurs de validation)

401 – Non autorisé (aucun utilisateur connecté alors qu'il devrait y en avoir un)

403 – L'utilisateur actuel n'est pas autorisé à accéder à cette donnée

404 – L'URL ne correspond à aucune route valide, ou la ressource demandée n'existe pas

405 – Méthode non autorisée (votre framework s'en chargera probablement automatiquement)

410 – La donnée a été supprimée, désactivée, suspendue, etc.

415 – Le type de contenu (Content-Type) de la requête n'est pas reconnu ou pris en charge par le serveur

429 – Limite de débit atteinte (Rate Limited), cela signifie : faites une pause, attendez un peu, puis réessayez

500 – Une erreur inattendue est survenue, c'est la faute de l'API

503 – L'API est temporairement indisponible, veuillez réessayer plus tard

Les formats de sortie et leurs type MIME (Multi-Purpose-INternet Mail Extensions)

Un type **MIME** représente le format de données transmises sur internet.

Un type **MIME (Multipurpose Internet Mail Extensions)** est une chaîne qui indique le format des données envoyées ou reçues via Internet.

Il est composé de deux parties type et sous-type, séparé par un / . (type/sous-type)

On distingue :

- **text/Html**
- **text/csv**
- **application/json**
- **application/xml**
- **image/jpeg**

b. Architecture REST : Principes, ressources, endpoints et structure des URLs

Comprendre les principes du REST

En matière de conception architecturale, qu'il s'agisse de génie civil ou de génie informatique, l'une des approches qui permettent de modéliser les systèmes complexes est toujours de partir de **ZÉRO**.

Conformément à ce point de vue, le concepteur part d'une feuille blanche et construit une architecture à partir de composants basiques et familiers jusqu'à ce qu'elle réponde aux besoins du système envisagé.

Le style architectural REST n'est pas en marge de cette approche car en effet, il a été développé par une application incrémentielle d'un ensemble de contraintes.

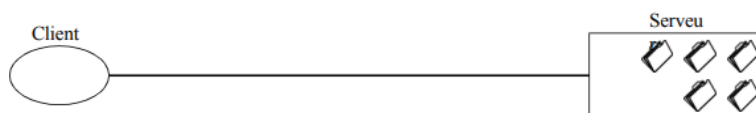
Le point de départ étant le **style Null** (ensemble vide de contraintes) en d'autres termes le style Null décrit un système dans lequel il n'y a pas de frontières entre les composants.

Contrainte 1 : Client-Serveur

Les premières contraintes ajoutées à notre **style Null** de départ sont celles du style architectural **client serveur**. Cette séparation des préoccupations est la première condition d'une architecture REST.

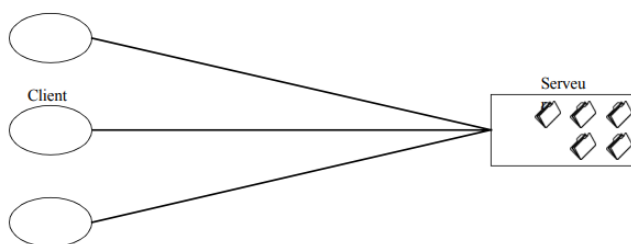
En séparant les préoccupations relatives à l'interface utilisateur des préoccupations relatives au traitement et stockage des données, on améliore la portabilité de l'interface utilisateur sur plusieurs plateformes et l'évolutivité en simplifiant les composants du serveur.

De plus, pour le web, cette claire séparation entre le client et le serveur permet aux composants d'évoluer indépendamment.



Contrainte 2 : Sans État (Stateless)

Nous ajoutons ensuite une contrainte à l'interaction client-serveur : la communication doit être sans état par nature, de sorte que chaque demande du client au serveur doit contenir toutes les informations nécessaires à l'exécution de la tâche.



L'état de la session est donc entièrement conservé sur le client.

Cette contrainte induit les propriétés de **visibilité**, de **fiabilité** et **d'évolutivité** :

La **visibilité** est améliorée parce qu'un système de surveillance n'a pas besoin de regarder au-delà d'une seule donnée de demande pour déterminer la nature complète de la demande.

La **fiabilité** est améliorée parce qu'elle facilite la récupération des défaillances partielles.

L'**évolutivité** est améliorée parce que le fait de ne pas avoir à stocker l'état entre les demandes permet au composant serveur de libérer rapidement des ressources, et simplifie encore la mise en œuvre parce que le serveur n'a pas à gérer l'utilisation des ressources entre les demandes.

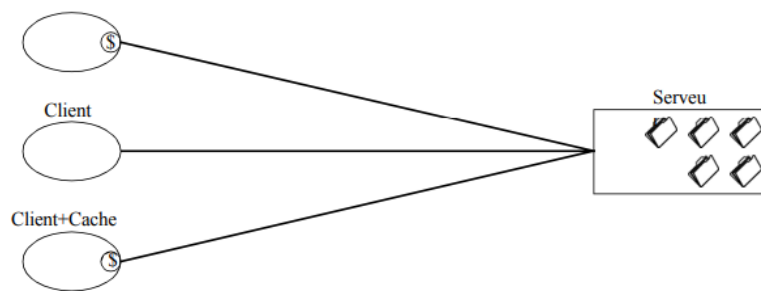
L'inconvénient est qu'elle peut diminuer les performances du réseau en augmentant les données répétitives (frais généraux par interaction) envoyées dans une série de requêtes, puisque ces données ne peuvent pas être laissées sur le serveur dans un contexte partagé.

Contrainte 3 : La mise en Cache

Afin d'améliorer l'efficacité du réseau, nous ajoutons des contraintes de cache pour former le style **client-cache-serveur sans état**.

Les contraintes de cache exigent que les données contenues dans une réponse à une requête soient implicitement ou explicitement étiquetées comme pouvant être mises en cache ou non.

Si une réponse peut être mise en cache, le cache du client a le droit de réutiliser les données de la réponse pour des demandes équivalentes ultérieures.



L'avantage de l'ajout de contraintes de cache est qu'il permet d'éliminer partiellement ou totalement certaines interactions, ce qui améliore l'efficacité, l'évolutivité et la performance perçue par l'utilisateur en réduisant la latence moyenne d'une série d'interactions.

Contrainte 4 : Uniformité de l'interface

La caractéristique centrale qui distingue le style **architectural REST** des autres styles basés sur les réseaux est l'accent mis sur une interface uniforme entre les composants.

Afin d'obtenir une interface uniforme, de multiples contraintes architecturales sont nécessaires pour guider le comportement des composants.

REST est défini par quatre contraintes d'interface : **l'identification des ressources, la manipulation des ressources par le biais de représentations, les messages auto-descriptifs et les hypertextes comme moteur de l'état de l'application.**

Contrainte 5 : Système en plusieurs niveaux

Contrainte 6 : Code à la demande

Le dernier ajout à notre ensemble de contraintes pour **REST** provient du style de code à la demande. **REST** permet d'étendre les fonctionnalités du client en téléchargeant et en exécutant du code sous forme d'applets ou de scripts.

REST vs SOAP

	API REST	SOA(Service-Oriented Architecture)
Définition	Modèle architectural conçu pour les services Web basés sur HTTP	Approche architecturale plus large basée sur l'intégration de services autonomes.
Principes fondamentaux	Basé sur les principes du REST : ressources, verbes HTTP , stateless.	Centré sur des principes comme les services indépendants et les contrats explicites.
Technologies utilisées	Principalement HTTP , JSON , XML , ou autres formats légers.	Utilise souvent des protocoles comme SOAP , WSDL , XML .
Granularité	Souvent plus fine, expose des ressources spécifiques.	Souvent plus grossière, expose des services complexes.
Facilité d'intégration	Plus simple grâce aux standards du Web.	Peut nécessiter plus d'efforts à cause des outils et protocoles spécifiques.
Performance	Haute performance grâce à sa légèreté et son optimisation HTTP .	Peut être plus lent à cause des surcharges liées à SOAP et XML .
Cas d'usage principal	Applications Web, microservices, systèmes mobiles.	Intégration d'applications complexes, systèmes d'entreprise (ERP , CRM).
Gestion de sécurité	Basée sur HTTPS , OAuth , JWT , etc.	Utilise des mécanismes comme WS-Security pour des scénarios complexes.

c. Format JSON : Structure des données échangées entre client et serveur.

Les données échangées entre le client – Serveur sont formalisées, afin de faciliter leur acheminement sur le réseau informatique. Cette opération s'appelle la **sérialisation**.

En définitive, la **sérialisation** est le processus qui consiste à transformer les données en un format **transmissible** ou **stockable**. Inversement la **désérialisation** transforme ce format en données exploitables.

Il existe de nombreux format de sérialisation – les principaux : **JSON** – **XML**

Le **JSON** repose sur deux éléments :

- **Objets (objects)** : Un objet est une liste de paires clé : valeur

```
{  
  "title" : "Le Petit Prince",  
  "author" : "Antoine de Saint-Exupéry",  
  "year" : 1943  
}
```

Ici, **title**, **author**, **year** = clés

Les valeurs peuvent être **texte**, **nombre**, **booléen**, **etc.**

- **Les tableaux (array)** : Un tableau est une liste de valeurs :

```
{  
  "books" : [  
    { "title" : "Livre 1" },  
    { "title" : "Livre 2" }  
  ]  
}
```

Lisibilité et **simplicité d'écriture** caractérise le **JSON**, de plus il y'a moins de texte donc plus rapide à transmettre (Légèreté).

d. Documentation openAPI/swagger : Lire et Interpréter une spécification pour en déduire les requêtes à construire

Considérons le besoin suivant, on souhaite créer une application qui permet de gérer le catalogue des livres appartenant à des catégories différentes.

Chaque **Livre** est défini par :

- Son **titre** de type String
- Son **auteur** de type String
- Son **description** de type float
- Sa **disponibilité** de type boolean

Une **catégorie** est définie par :

- Son **code** de type Long (Auto Increment)
- Son **nom** de type String

L'application doit permettre :

- D'ajouter une nouvelle catégorie
- Ajouter un livre appartenant à une catégorie
- Consulter toutes les catégories
- Consulter les livres d'une catégorie
- Consulter un livre
- Mettre à jour un livre
- Supprimer une catégorie

Voir la spécification fonctionnelle détaillée **specs.yml** : [click here](#)

e. Travaux-pratiques :

- Exploration d'une API du domaine public

3. Structuration des requêtes - Utilisation des variables - Scripts et automatisation

Objectifs :

1. Utilisation des variables - Scripts et automatisation
2. Variables d'environnement : Isoler les configurations spécifiques à chaque contexte (dev, recette, production) Variable de collection et variable globale : portée et priorité de résolution des différents scopes.
3. Données dynamiques : Générer des valeurs aléatoires et horodatées avec les variables intégrées de Postman.
4. Variabilisation des requêtes : Remplacer les valeurs en dur par des références de variables pour rendre les collections portables.
5. Ecriture Scripts post response de test pour valider les réponses (status code, body, headers, temps de réponse)
6. utiliser la console Postman pour inspecter les requêtes, réponses.
7. Chaînage de requêtes.

Atelier :

Name : Simple Books API

Description : Cette API permet de réserver des livres.

url : <https://simple-books-api.click>

Endpoints

Status

GET /status : Returns the status of the API.

List of books

GET /books : Returns a list of books.

Optional query parameters:

type: fiction or non-fiction

limit: a number between 1 and 20.

Get a single book

GET /books/:bookId : Retrieve detailed information about a book.

Submit an order

POST /orders : Allows you to submit a new order. Requires authentication.

The request body needs to be in JSON format and include the following properties:

bookId - Integer - Required

customerName - String - Required

Example

POST /orders/

Authorization: Bearer <YOUR TOKEN>

```
{  
  "bookId": 1,  
  "customerName": "John"  
}
```

The response body will contain the access token.

Get all orders

GET /orders : Allows you to view all orders. Requires authentication.

Get an order

GET /orders/:orderId : Allows you to view an existing order. Requires authentication.

Update an order

PATCH /orders/:orderId : Update an existing order. Requires authentication.

The request body needs to be in **JSON** format and allows you to update the following properties:

customerName - String

Example

PATCH /orders/PF6MflPDcuhWobZcgmJy5

Authorization: Bearer <YOUR TOKEN>

```
{  
  "customerName": "John"  
}
```

Delete an order

DELETE /orders/:orderId : Delete an existing order. Requires authentication.

The request body needs to be empty.

Example

DELETE /orders/PF6MflPDcuhWobZcgmJy5

Authorization: Bearer <YOUR TOKEN>

API Authentication

To submit or view an order, you need to register your **API client**.

POST /api-clients/

The request body needs to be in **JSON** format and include the following properties:

clientName - String

clientEmail - String

Example

```
{  
  "clientName": "Valentin",  
  "clientEmail": "valentin@example.com"  
}
```

The response body will contain the access token.

Possible errors

Status code 409 - "API client already registered." Try changing the values for **clientEmail** and **clientName** to something else.

PLAN DE TEST AUTOMATISÉ POSTMAN – Simple Books API

Environnement Postman

Variables à définir (dans l'environnement) :

Nom	Description	Exemple
baseUrl	URL de base de l'API	https://simple-books-api.glitch.me
accessToken	Token d'authentification	(rempli dynamiquement)
orderId	ID d'une commande créée	(rempli dynamiquement)
clientEmail	Email unique pour tests	test+{{timestamp}}@mail.com
timestamp	Timestamp dynamique pour éviter 409	pm.environment.set(...)
bookId	ID d'un livre	(récupéré dynamiquement)

POST /api-clients/ – Obtenir un accessToken

Requête

POST {{baseUrl}}/api-clients/

Body (JSON) :

```
{  
  "clientName": "Postman Test",  
  "clientEmail": "test+{{timestamp}}@mail.com"  
}
```

Pré-Request Script :

```
pm.environment.set("timestamp", Date.now());
```

Post-Response

```
let jsonData = pm.response.json();  
pm.test("Token reçu", function () {
```

```
pm.expect(jsonData.accessToken).to.exist;
});
pm.environment.set("accessToken", jsonData.accessToken);
```

GET /status – Vérifier l'état de l'API

Requête

GET {{baseUrl}}/status

Post-Response

```
pm.test("API opérationnelle", function () {
  pm.expect(pm.response.code).to.equal(200);
});
```

GET /books – Liste des livres

Requête

GET {{baseUrl}}/books

Post-Response

```
let books = pm.response.json();
pm.test("Liste non vide", function () {
  pm.expect(books.length).to.be.above(0);
});
```

```
// Stocker le premier bookId
pm.environment.set("bookId", books[0].id);
```

GET /books/:bookId – Détails d'un livre

Requête

GET {{baseUrl}}/books/{{bookId}}

Post-Response

```
let book = pm.response.json();
pm.test("Détail du livre OK", function () {
  pm.expect(book.id).to.equal(parseInt(pm.environment.get("bookId")));
});
```

POST /orders – Créer une commande

Requête

POST {{baseUrl}}/orders

Header :

Authorization: Bearer {{accessToken}}

Body :

```
{
  "bookId": {{bookId}},
```

```
"customerName": "Jean Dupont"  
}
```

```
let jsonData = pm.response.json();  
pm.test("Commande créée", function () {  
    pm.expect(jsonData.orderId).to.exist;  
});  
pm.environment.set("orderId", jsonData.orderId);
```

GET /orders – Voir toutes les commandes

Requête

GET {{baseUrl}}/orders

Header :

Authorization: Bearer {{accessToken}}

Test

```
pm.test("Requête autorisée", function () {  
    pm.expect(pm.response.code).to.equal(200);  
});
```

GET /orders/:orderId – Voir une commande

Requête

GET {{baseUrl}}/orders/{{orderId}}

Header :

Authorization: Bearer {{accessToken}}

Post-response

```
let order = pm.response.json();  
  
pm.test("Commande correcte", function () {  
    pm.expect(order.id).to.equal(pm.environment.get("orderId"));  
    pm.expect(order.customerName).to.equal("Jean Dupont");  
});
```

PATCH /orders/:orderId – Modifier une commande

Requête

PATCH {{baseUrl}}/orders/{{orderId}}

Header :

Authorization: Bearer {{accessToken}}

Body :

```
{  
  "customerName": "Nouveau Nom"  
}
```

DELETE /orders/:orderId – Supprimer une commande

Requête

DELETE {{baseUrl}}/orders/{{orderId}}

Header :

Authorization: Bearer {{accessToken}}

Body : vide

Test

```
pm.test("Commande supprimée", function () {  
  pm.expect(pm.response.code).to.equal(204);  
});
```

4. Newman pour automatiser l'exécution des tests

Newman est l'outil en ligne de commande de Postman pour lancer des collections.

Installer Newman

npm install -g newman

Installer un reporter HTML

Le plus courant est newman-reporter-htmlextra :

npm install -g newman-reporter-htmlextra

Le dépôt du reporter indique qu'il s'active avec **-r htmlextra**.

Exécuter une collection et produire le HTML

newman run simple-book-api.postman_collection.json -r cli,htmlextra

Cette commande génère un rapport HTML dans le dossier **./newman** par défaut.

Choisir le nom et l'emplacement du rapport

**newman run simple-book-api.postman_collection.json -r cli,htmlextra
--reporter-htmlextra-export reports/rapport.html**

Exécution avec un environnement Postman

```
newman run simple-book-api.postman_collection.json -e mon-environnement.json -r  
cli,htmlextra --reporter-htmlextra-export reports/rapport.html
```